

Lecture 5: Compound Types and Iteration

Laboratory: Coding - Accounting, Finance and Business Consulting

Data Types in this Lecture - Compound Types

Type	Description	Purpose	Scalar	Mutability	Order
<code>list</code>	Mutable ordered collection of items	Storing items in a changeable sequence	Non-scalar	Mutable	Ordered (sequence)
<code>tuple</code>	Immutable ordered collection of items	Grouping related data together	Non-scalar	Immutable	Ordered (sequence)
<code>set</code>	Unordered collection of unique items	Removing duplicates and membership tests	Non-scalar	Mutable	Unordered
<code>dict</code>	Unordered collection of key-value pairs	Storing and manipulating keys-values pairs	Non-scalar	Mutable	Unordered (but keyed)

- Lists and tuples are both ordered collections
 - Lists are mutable (i.e., can be modified)
 - Tuples are immutable
- Sets and dictionaries are both unordered
 - Sets only store individual items
 - Dictionaries store key-value pairs

We will look more closely to lists, a bit to sets and just a quick glance to tuples and dictionaries

Non-Scalar Data Types

- String – sequence of characters (immutable, ordered)
- List – sequence of values (mutable, ordered)
- Set – set of unique values (mutable, unordered)
- Tuple – sequence of values (immutable, ordered)
- Dictionary – a set of key/value pairs (mutable, unordered)

Get familiar with how they look when printed:

Non-Scalar Data Types - Appearance

```
In [1]: strng = "ciao"
print(f"strng is a string data type and it prints: \n{strng}")

lst = [1, 2, 2, 3, 4]
print(f"lst is a list containing integers and it prints: \n{lst}")

st = {1, 2, 2, 3, 4}
print(f"st is a set, which removes duplicates, and it prints: \n{st}")

tpl = (1, 2, 2, 3, 4, 5)
print(f"tpl is a tuple, and it prints: \n{tpl}")

dct = {'a': 1, 'b': 2}
print(f"dct is a dictionary with key-value pairs, and it prints: \n{dct}")
```

```
strng is a string data type and it prints:
ciao
lst is a list containing integers and it prints:
[1, 2, 2, 3, 4]
st is a set, which removes duplicates, and it prints:
{1, 2, 3, 4}
tpl is a tuple, and it prints:
(1, 2, 2, 3, 4, 5)
dct is a dictionary with key-value pairs, and it prints:
{'a': 1, 'b': 2}
```

Lists

Lists

Lists are defined using square brackets with elements separated by commas:

```
[ element_at_index_0, ..., element_at_index_n ]
```

- A list allows to store multiple elements
 - `element_at_index_0` and `element_at_index_n`.
- Lists can **contain a mix of any type** of data
 - Can also contain other lists (nested list)
- Usually, we will use lists to store elements of the same type
 - Allows to perform operations on the entire list at once
 - Applying operations to a list that contains different types of data need extra care
 - Not advisable to use mixed-type list if you need to do substantial data manipulation
 - Still, if you need to store different types of data, lists are the most flexible option

Lists - Some Simple Examples

```
In [2]: list_ex_numbers = [4, 2, 3]
print(f"list_ex_numbers is a list of integers and it prints: {list_ex_numbers}")

list_ex_strings = ['elephant', 'human']
print(f"list_ex_strings is a list of strings and it prints: {list_ex_strings}")

list_ex_mix = [0, 0.3, 'string']
print(f"list_ex_mix is a list of mixed data types and it prints: {list_ex_mix}")

list_ex_nested = [list_ex_numbers, list_ex_strings, list_ex_mix]
print(f"list_ex_nested is a list containing other lists and it prints: \n{list_ex_nested}")
```

list_ex_numbers is a list of integers and it prints: [4, 2, 3]

list_ex_strings is a list of strings and it prints: ['elephant', 'human']

list_ex_mix is a list of mixed data types and it prints: [0, 0.3, 'string']

list_ex_nested is a list containing other lists and it prints:
[[4, 2, 3], ['elephant', 'human'], [0, 0.3, 'string']]

Empty List

A list can be empty, e.g., created with

```
list()
```

or

```
[]
```

Useful when you need some place to store data

- Gives the most flexibility
- You can add elements to the list later, we will see this is very useful for iteration

Empty List

```
In [3]: list_ex_empty = list()
print(f"list_ex_empty is an empty list created using the list() constructor")

list_ex2_empty = []
print(f"list_ex2_empty is an empty list created using square brackets and it prints: []")
```

list_ex_empty is an empty list created using the list() constructor and it prints: []

list_ex2_empty is an empty list created using square brackets and it prints: []

Creating Lists Using the Asterisk (*) and Plus (+) Operators in Python

Bullet Points:

- The asterisk `*` operator can be used to create lists with repeated elements
 - Syntax for asterisk: `[element] * n`, where `element` is the item to repeat and `n` is the number of repetitions
- The plus `+` operator is used for list concatenation.
 - Syntax for plus: `[list1] + [list2]`, where `list1` and `list2` are the lists to concatenate
- Both operators can be combined for more complex list creations
- Useful for initializing lists, extending lists, or creating patterned lists

```
In [4]: # Create a list of zeros with length 5 using *
zeros_list = [0] * 5
print(f"List of zeros with length 5 using *: \n{zeros_list}")
# Create a list combining * and + operators
mixed_list = [1, 2, 3] + [0] * 2
print(f"List combining * and + operators: \n{mixed_list}")
# Extend a list using + operator
extended_list = ["Hi", "Ciao", "Ola"] + [4, 5, 6]
print(f"Extended list using + operator: \n{extended_list}")
# Create a 2D list (list of lists) using * operator
two_d_list = [[0] * 3] * 2
print(f"2D list using * operator: \n{two_d_list}")
```

List of zeros with length 5 using *:

[0, 0, 0, 0, 0]

List combining * and + operators:

[1, 2, 3, 0, 0]

Extended list using + operator:

['Hi', 'Ciao', 'Ola', 4, 5, 6]

2D list using * operator:

[[0, 0, 0], [0, 0, 0]]

Exercises on Creating Lists Using the Asterisk (*) and Plus (+) Operators

1. Initialize a List with Quarterly Revenue

Create a list that contains four identical quarterly revenues of \$10,000.

2. Concatenate Lists of Quarterly Expenses and Revenues

Create a list by concatenating two lists, one containing quarterly expenses

`[$2000, $3000]` and another containing quarterly revenues `[$5000, $6000]`.

3. List of Interest Rates for Six Months

Create a list that has alternating interest rates of 1% and 2%, starting with 1%, for a year.

4. Nested Lists of Monthly Sales for Three Departments

Create a list that contains three sub-lists, each containing the monthly sales for three departments. Each department has identical monthly sales of \$2000.

5. List of Economic Indicators

Create a list that starts with two GDP growth rates of 3%, followed by three inflation rates of 2%, and ends with a single employment rate of 95%.

Lists Are Ordered

- Similar to strings, lists are ordered collections of elements
- We can use the `[]` operator to
 - **index** to retrieve a single element
 - **slice** to retrieve a sublist
 - **stride** slicing and striding for selecting elements of a sublist in a stepwise manner
- Let's have a quick refresher

Non-negative indexes

Non-negative indexes are 0 or positive integers; 0 indicates the first element, reading from left to right

On a list defined as `["Ann", "Ben", "Cho"]` it is:

Non-negative index:	0	1	2 (i.e., <code>len(list)-1</code>)
Element:	Ann	Ben	Cho

Negative indexes

Negative indexes are negative integers; -1 indicates the last element, reading from left to right. See below.

Negative index:	-3 (i.e., <code>-len(list)</code>)	-2	-1
Element:	Ann	Ben	Cho

Indexing a Nested List

- Lists can contain other lists
- When you extract a nested list using indexing you get a list

```
In [5]: nested_list_1 = [  
        ['Hermione' , 'A'],
```

```

        ['Harry Potter', 'B'],
        ['Ronald' , 'C']
    ]

```

```
print(f"nested_list_1 is: {nested_list_1}")
```

```
nested_list_1 is: [['Hermione', 'A'], ['Harry Potter', 'B'], ['Ronald', 'C']]
```

```
In [6]: print(f"Print the first element of nested_list_1: {nested_list_1[0]}")
        print(f"Print the first element of nested_list_1 using negative indexing: {r
```

```
Print the first element of nested_list_1: ['Hermione', 'A']
```

```
Print the first element of nested_list_1 using negative indexing: ['Hermione', 'A']
```

Indexing a Nested List

- By using **nested indexing** you can access elements of **nested lists**
 - Nested indexing applies indexing multiple times
 - E.g., take the n-th element of the n-th element of a list

```
In [7]: print(f"nested_list_1 is: {nested_list_1}")
        print(f"Print the first element of the first element of nested_list_1 using nested indexing: {nested_list_1[0][0]}")
        print(f"Print the last element of the first element of nested_list_1 using nested indexing: {nested_list_1[0][-1]}")
        print(f"Print the last element of the last element of nested_list_1 using nested indexing: {nested_list_1[-1][-1]}")
```

```
nested_list_1 is: [['Hermione', 'A'], ['Harry Potter', 'B'], ['Ronald', 'C']]
```

```
Print the first element of the first element of nested_list_1 using nested indexing: Hermione
```

```
Print the last element of the first element of nested_list_1 using nested indexing: A
```

```
Print the last element of the last element of nested_list_1 using nested indexing: C
```

🏆 Exercises on Indexing and Nested Lists in Python

1. Retrieve the First Quarter Revenue for Department A

Given a nested list of quarterly revenues for three departments A, B and C

```
[[10000, 12000, 11000, 10500], [9000, 9500, 9200, 9300], [8500, 8700, 8600, 8800]]
```

, retrieve the first quarter revenues for Department A.

2. Find the Last Month's Expense for the Company

Given an ordered list of monthly expenses by quarter `[[2000, 2100, 2050, 2200], [3000, 3100, 3050, 3010], [2000, 5200, 5020, 5070]]`, find the expenses for December.

3. Retrieve the Interest Rate for the Third Month of the First Loan

You have a nested list of monthly interest rates for two different loans `[[1.5,`

```
2.1, 1.2, 2.7, 1.3, 2.8], [2.9, 1.0, 2.0, 1.1, 2.5, 1.6]] .
```

Retrieve the interest rate for the third month of the first loan.

Lists are Mutable

So we can use Indexing, slicing and striding to update elements

Update an Element via Indexing

We can update a single element at a given index as in the following example.

```
In [8]: slides_list = [ 'Harry', 'Hermione', 'Nevil', 'Malfoy', 'Sirius' ]
print(f'Before updating the list is: {slides_list}')

# update the element at index 0
slides_list[0] = 'Ronald'
print(f'After updating the list is: {slides_list}')
```

Before updating the list is: ['Harry', 'Hermione', 'Nevil', 'Malfoy', 'Sirius']

After updating the list is: ['Ronald', 'Hermione', 'Nevil', 'Malfoy', 'Sirius']

Update Several Elements via Slicing

- We can update several elements at once using slicing
- From a given index (included) up to another (excluded)

```
In [9]: slides_list = [ 'Harry', 'Hermione', 'Nevil', 'Malfoy', 'Sirius' ]
print(f'Before updating the list is:\n{slides_list}')

# update the elements at index 1 and 2
slides_list[ 1 : 3 ] = [ 'Ginny', 'Luna' ]
print(f"\nAfter updating the list using the command:\nslides_list[ 1 : 3 ] =
```

Before updating the list is:

['Harry', 'Hermione', 'Nevil', 'Malfoy', 'Sirius']

After updating the list using the command:

slides_list[1 : 3] = ['Ginny', 'Luna']

the list becomes:

['Harry', 'Ginny', 'Luna', 'Malfoy', 'Sirius']

Update Several Elements via Slicing and Striding

- We can selectively update via slicing and striding
- From a given index (included) up to another (excluded) in steps

```
In [10]: slides_list = [ 'Harry', 'Hermione', 'Nevil', 'Malfoy', 'Sirius', 'Gary' ]
print(f"Before updating the list is:\n{slides_list}")
```

```
# update all list elements at even indexes
slides_list[::2] = [ 'Ginny', 'Luna', 'Chris' ]
print(f"\nAfter updating the list using the command:\nslides_list[::2] = [ 'Ginny', 'Luna', 'Chris' ]")
```

Before updating the list is:

```
['Harry', 'Hermione', 'Nevil', 'Malfoy', 'Sirius', 'Gary']
```

After updating the list using the command:

```
slides_list[::2] = [ 'Ginny', 'Luna', 'Chris' ]
```

the list becomes:

```
['Ginny', 'Hermione', 'Luna', 'Malfoy', 'Chris', 'Gary']
```

Delete Several Elements

We can delete several elements at once by assigning the empty list to a slice

```
In [11]: slides_list = [ 'Harry', 'Hermione', 'Nevil', 'Malfoy', 'Sirius' ]
print(f"Before deleting the list is:\n{slides_list}")# remove from index 1 to 3
slides_list[1:3] = []
print(f"\nAfter deleting elements using the command:\nslides_list[1:3] = []\nslides_list[:] = []# empty the list")
print(f"\nAfter emptying the list using the command:\nslides_list[:] = []\n")
```

Before deleting the list is:

```
['Harry', 'Hermione', 'Nevil', 'Malfoy', 'Sirius']
```

After deleting elements using the command:

```
slides_list[1:3] = []
```

the list becomes:

```
['Harry', 'Malfoy', 'Sirius']
```

After emptying the list using the command:

```
slides_list[:] = []
```

the list becomes:

```
[]
```

List Methods - Append, Insert and Remove

- `L.append(e)` adds the element `e` to the end of the list `L`
- `L.insert(i, e)` inserts the element `e` at index `i` in the list `L`
- `L.remove(e)` removes the first occurrence of `e` from the list `L`

List Methods - Append, Insert and Remove

```
In [12]: # Demonstrating the list methods with examples
L = [3, 1, 4, 1, 5, 9]
print(f"Initial list: {L}\n")
# Using append
L.append(2)
print(f"After appending 2: {L}\n")
# Using insert
L.insert(2, 6)
```

```
print(f"After inserting 6 at index 2: {L}\n")
# Using remove
L.remove(1) # removing the first occurrence of 1
print(f"After removing the first occurrence of 1: {L}\n")
#using pop – remove and display based on index
L.pop(3)
print(f"After removing the fourth element: {L}")
```

Initial list: [3, 1, 4, 1, 5, 9]

After appending 2: [3, 1, 4, 1, 5, 9, 2]

After inserting 6 at index 2: [3, 1, 6, 4, 1, 5, 9, 2]

After removing the first occurrence of 1: [3, 6, 4, 1, 5, 9, 2]

After removing the fourth element: [3, 6, 4, 5, 9, 2]

List Methods - Index

- `list_name.index(value)` returns the index of the first occurrence of the specified value in a list
 - If the value is not found it raises a `ValueError`
- Useful if you need to identify the position of an observation of interest

```
In [13]: # Using Index
L2 = ["Italy", "France", "US", "China"]
print(f"The new list L2 is: {L2}\n")
print(f"Using L2.index('China') the returned index is: {L2.index('China')}\n")
print(f"Using L2.index('Vietnam') leads to an error")
```

The new list L2 is: ['Italy', 'France', 'US', 'China']

Using `L2.index('China')` the returned index is: 3

Using `L2.index('Vietnam')` leads to an error

Be on the Lookout for one-element Lists

- It is easy to confuse a one-element list with a scalar:
 - `['Hagrid']` vs `'Hagrid'`
- Common mistake: Use a scalar method to a list getting **"inexplicable"** errors

```
In [14]: slides_list = ['Harry', 'Hermione', 'Nevil', 'Malfoy', 'Sirius']
print('before appending', slides_list)
slides_list.append(['Hagrid'])
print('after appending ', slides_list)
print('Hagrid in slides_list) #for example you won't find Hagrid
slides_list.upper() #and you cannot use string methods on it
```

```
before appending ['Harry', 'Hermione', 'Nevil', 'Malfoy', 'Sirius']
after appending ['Harry', 'Hermione', 'Nevil', 'Malfoy', 'Sirius', ['Hagrid']]
False
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[14], line 6
      4 print('after appending ', slides_list)
      5 print('Hagrid' in slides_list) #for example you won't find Hagrid
----> 6 slides_list.upper() #and you cannot use string methods on it

AttributeError: 'list' object has no attribute 'upper'
```

List Methods - Extend and Sort

- `L.sort()` sorts the elements of list `L` in ascending order
 - If the elements are **numbers**, `.sort()` uses the standard order among numbers.
 - If the elements are **strings**, `.sort()` uses the lexicographical order (the order used in dictionaries) and upper-case letters precede lower-case letters
- `L.extend(L1)` adds the elements of list `L1` to the end of the list `L` (notice the difference with `append`)

For a complete list of methods, see [here](#)

List Methods - Sort

```
In [15]: # Using sort
L.sort()
print(f"After sorting: {L}\n")

# List of strings
L_string_notSort = ['c', 'a', 'b', 'C', 'A', 'B']

# Store the unsorted list before sorting
# This is needed because .sort(), and methods in general, tend to modify the
L_string_sortMethod = L_string_notSort.copy()
# Sorting the list
L_string_sortMethod.sort()

# There are usually functions that are analogous to methods that directly re
L_string_sortFunc = sorted(L_string_notSort)

# Showing both the unordered and ordered versions
print(f"The unordered list of strings: {L_string_notSort}\n")
print(f"The ordered list of strings after sorting using the method .sort():\n")
print(f"The ordered list of strings after sorting using the function sorted(
```

After sorting: [2, 3, 4, 5, 6, 9]

The unordered list of strings: ['c', 'a', 'b', 'C', 'A', 'B']

The ordered list of strings after sorting using the method `.sort()`:
['A', 'B', 'C', 'a', 'b', 'c']

The ordered list of strings after sorting using the function `sorted()`:
['A', 'B', 'C', 'a', 'b', 'c']

List Methods - Extend

```
In [16]: # Partly modified list
print(f"Partly modified list: {L}\n")

# Using extend
L1 = [8, 0, 7]
L.extend(L1)
print(f"After extending with {L1}: {L}\n")

# Using append
L.append(L1)
print(f"After appending with {L1}: {L}\n")
```

Partly modified list: [2, 3, 4, 5, 6, 9]

After extending with [8, 0, 7]: [2, 3, 4, 5, 6, 9, 8, 0, 7]

After appending with [8, 0, 7]: [2, 3, 4, 5, 6, 9, 8, 0, 7, [8, 0, 7]]

Function and Methods - Creation of New Variable or In-Place Overwriting

- Usually functions create a new object, while methods modify their object
- E.g., the method `L.sort()` sorts the elements of `L` in place
- If you want to keep the original list, you can use the function `sorted(L)`
- A very wise policy is to **always** check whether the method you are using is overwriting or not

```
In [17]: myList = [4, 5, 2, 1, 3]
print(f"This is myList: {myList}")
myList.sort() # Sorts in-place. It is more efficient but overwrites the input
print(f"This is mylist after using myList.sort(): {myList}\n")

myOtherList = [10, 9, 6, 8, 7]
print(f"This is myOtherList: {myOtherList}")
print(f"this is the result of sorted(myOtherList): {sorted(myOtherList)}")
print(f"This is myOtherList after running sorted(myOtherList): {myOtherList}")
```

This is myList: [4, 5, 2, 1, 3]

This is myList after using myList.sort(): [1, 2, 3, 4, 5]

This is myOtherList: [10, 9, 6, 8, 7]

this is the result of sorted(myOtherList): [6, 7, 8, 9, 10]

This is myOtherList after running sorted(myOtherList): [10, 9, 6, 8, 7]

Useful Functions for Compound Types

- General
 - `len()` – returns the length of the sequence or collection

```
In [18]: data = [70, 80, 50, 60, 90]
print(f"We will use this list of numbers: {data}\n")

length = len(data)
print(f"Length of the list is: {length}")
```

We will use this list of numbers: [70, 80, 50, 60, 90]

Length of the list is: 5

Useful Mathematical Functions for Compound Types

- `min()` – returns the smallest element
- `max()` – returns the largest element
- `sum()` – returns the sum of all elements

```
In [19]: # Mathematical functions
minimum = min(data)
print(f"The smallest element in the list is: {minimum}")

maximum = max(data)
print(f"The largest element in the list is: {maximum}")

total = sum(data)
print(f"The sum of all elements in the list is: {total}")
```

The smallest element in the list is: 50

The largest element in the list is: 90

The sum of all elements in the list is: 350

Exercises on Lists Methods

1. You've collected monthly sales data for a product: [1200, 1300, 1100, 1150]. Calculate the total number of months for which you have data. How many months of data have you collected?
2. From a dataset of daily website visitors: [105, 120, 130, 125, 110, 90], identify the day with the lowest visitors. How many visitors did you have on that day?

3. Using the same dataset of daily website visitors, identify the day with the highest number of visitors. How many visitors did you have on that day?
4. (Note: Python doesn't have a built-in `mean()` function. Instead, you can use `sum(list)/len(list)` to calculate the mean.) From the sales data: `[1200, 1300, 1100, 1150]`, calculate the average monthly sales. What's the average sales figure?
5. You have a list of monthly savings: `[200, 220, 210, 230]`. Calculate the total savings over these months. How much have you saved in total?
6. You have a list of product ratings given by customers: `[3.5, 4.2, 3.8, 4.5, 4.0]`. Sort the ratings to see the range of feedback. What does the sorted list look like?
7. You've been tracking the yearly profits from 2023 to 2019: `[2023, 2022, 2021, 2020, 2019]`. For a report, you need the data in ascending order. Arrange the years in order. What does the ordered list look like?

Iteration

Iteration

- We will now see how iteration works since it is the prime way to perform operations on lists
 - Its flexibility makes it one of the most important tool in coding at large
- Iteration allows to perform **a set of operations on multiple inputs**
 - Repeating the same code on all the:
 - Elements of a list, columns of a table, different datasets, etc.
- You could simply copy-and-paste the code, but this is:
 - **Sometimes unfeasible** (e.g. if you have a list of 1000 elements)
 - **Error-prone and boring**
 - **Hard to understand and mantain**

Avoiding Code Duplication Reduces Bugs

- Reducing code duplication has three main benefits:
 1. It's easier to see the **intent** of your code
 - If you have a chunk of code of 20 lines to be applied to 100 elements, it's hard to see what 2000 lines of code are doing
 2. It's easier to respond to **changes** in requirements
 - e.g. if you have to change the code, you have to change it in 100 places instead of 1

3. You're likely to have **fewer bugs**

- Computers do not do mistakes in repetition, humans do

For Loops in Python

- In Python, loops are defined over iterables
- Iterables are objects that can be iterated over: sequences (lists, dataframes, etc), iterators

For Loops in Python

```
for value in iterable:           #Definition of the sequence  
    do_something_with_value      #Body
```

- `for value in iterable:` defines the `value` variable that assumes **sequentially** each value in the iterable and executes the code in the body of the loop
 - You can use whatever name instead of `value`
 - Common to use `iterable_loop`
 - i.e., if the iterable is named `listXXX`, the value name is `listXXX_loop`
- The **body of the loop is indented** and contains the code to be executed for each value in the iterable

For Loops in Python

```
In [20]: print(f"Simple for loop over a list\n") #describe what the code below does  
L = [1, 2, 3, 4, 5] #create a list  
for val in L: #iterate over the list  
    print(f"val = {val}") #print the value of the current iteration
```

Simple for loop over a list

```
val = 1  
val = 2  
val = 3  
val = 4  
val = 5
```

Nested For Loops

- Loops can be combined to create nested loops
- In the simplest setting of two loops there is an **outer loop** and an **inner loop**
- The outer loop is going to iterate across its values
 - For each value on the outer loop the inner loop is going to iterate across its values
- E.g. you have two lists of space (north, south, east, west) and time (2020, 2021, etc.) you can use a nested loop to consider all the possible combinations i.e. the

cartesian product

- Watch out for indentation!

```
In [21]: # Data lists for spatial directions and years
space = ['North', 'South', 'East', 'West']
years = [2020, 2021, 2022]
print(f"We have two lists: space {space} and years {years}")
print("We will use nested for loops to generate all possible combinations of")
# Nested For Loops to generate combinations
for location in space:
    for year in years: #Single INDENTED! | is inside 1 for
        # Generate and print each space-time combination | DOUBLE INDENTED-
        print(f"Location: {location}, Year: {year}")
```

We have two lists: space ['North', 'South', 'East', 'West'] and years [2020, 2021, 2022]

We will use nested for loops to generate all possible combinations of space and time

```
Location: North, Year: 2020
Location: North, Year: 2021
Location: North, Year: 2022
Location: South, Year: 2020
Location: South, Year: 2021
Location: South, Year: 2022
Location: East, Year: 2020
Location: East, Year: 2021
Location: East, Year: 2022
Location: West, Year: 2020
Location: West, Year: 2021
Location: West, Year: 2022
```

for Loops over Strings

- Iteration can also be performed on strings looping over its characters
- We will see that loops can be used virtually on everything

```
In [22]: my_string = 'hello'
print(f"my_string is: {my_string}")
for element in my_string:
    print(f'The string has the "{element}" character')
```

```
my_string is: hello
The string has the "h" character
The string has the "e" character
The string has the "l" character
The string has the "l" character
The string has the "o" character
```

The range Function

- Generates a sequence of numbers
 - You can access its elements using indexing, slicing and striding

- You can convert it to Lists, tuples, etc.
- Commonly used to set the values of a `for` loop
- Syntax: `range(start, stop, step)`
 - `start` : Starting value (default is 0)
 - `stop` : Ending value (exclusive)
 - `step` : Increment (default is 1)
 - If called with just one par it is assumed to be the `stop`
 - `range(5)` is the same as `range(0, 5, 1)`
 - If called with two pars they are assumed to be `start` and `stop`
 - `range(2, 4)` is the same as `range(2, 4, 1)`

The `range` Function

- Generates a sequence of numbers
 - You can access its elements using indexing, slicing and striding
 - You can convert it to Lists, tuples, etc.
 - Commonly used to set the values of a `for` loop
- Syntax: `range(start, stop, step)`
 - `start` : Starting value (default is 0)
 - `stop` : Ending value (exclusive)
 - `step` : Increment (default is 1)

The `range` Function as an Iterable

```
In [23]: print(f"List of range(5): {list(range(5))}")
print(f"First element of range(5), range(5)[0]: {range(5)[0]}")
print(f"Last element of range(2, 4),range(2, 4)[-1]: {range(2, 4)[-1]}")
print(f"List from range(0, 11, 2): {list(range(0, 11, 2))}")
```

```
List of range(5): [0, 1, 2, 3, 4]
First element of range(5), range(5)[0]: 0
Last element of range(2, 4),range(2, 4)[-1]: 3
List from range(0, 11, 2): [0, 2, 4, 6, 8, 10]
```

The `range` Function in Loops

```
In [24]: for i in range(5):
print(f"Value from range(5): {i}")
print("\n")
for i in range(2, 4):
print(f"Value from range(1, 6): {i}")
print("\n")
for i in range(0, 11, 2):
print(f"Value from range(0, 11, 2): {i}")
```

```
Value from range(5): 0
Value from range(5): 1
Value from range(5): 2
Value from range(5): 3
Value from range(5): 4
```

```
Value from range(1, 6): 2
Value from range(1, 6): 3
```

```
Value from range(0, 11, 2): 0
Value from range(0, 11, 2): 2
Value from range(0, 11, 2): 4
Value from range(0, 11, 2): 6
Value from range(0, 11, 2): 8
Value from range(0, 11, 2): 10
```

Iterating Through Lists with `for` Loop Using Index and Element

- Using `range(len(iterable))` to create the loop sequence you can access both indexes and elements
 - `len(iterable)` : Returns the length of the iterable
 - `range()` : Generates indices from 0 to `len(iterable)-1`
 - You can use the indices to access the elements of the iterable
- Useful for tasks that require element manipulation based on index
 - Modify elements based on their index
 - Use the index along with the element

```
In [25]: # Use range and len to get both index and element
fruits = ['apple', 'banana', 'cherry']
print(f"Original List: {fruits}\n") # print original list

# For Loop using range and len to iterate over the list
print(f"Using range and len to iterate over the list")
for index in range(len(fruits)):
    element = fruits[index] # Access element using index
    print(f"Index: {index}, Element: {element}") # print index and element
```

```
Original List: ['apple', 'banana', 'cherry']
```

```
Using range and len to iterate over the list
Index: 0, Element: apple
Index: 1, Element: banana
Index: 2, Element: cherry
```

Iterating Through Lists with `for` Loop Using Enumerate

- `enumerate()` is a more efficient and more readable alternative that allows to access both indexes and elements of a sequence

- Syntax `enumerate(iterable, start=0)`
 - `iterable` : The sequence to iterate through
 - `start` : The starting index (default is 0)

```
In [26]: print(f"\nUsing enumerate to iterate over the list getting index and element")
for index, element in enumerate(fruits): # Using enumerate to get both index
    print(f"Index: {index}, Element: {element}") # print index and element
```

```
Using enumerate to iterate over the list getting index and element
Index: 0, Element: apple
Index: 1, Element: banana
Index: 2, Element: cherry
```

Exercises on Python Iteration

1. Basic Loop with Range

- Write a script that prints the numbers from 1 to 10 using a `for` loop and the `range` function.

2. Even Numbers with Range

- Write a script that prints all even numbers between 1 and 20 using a `for` loop and the `range` function (if you are wondering how to identify even numbers from odd ones, have a refresh of the third set of slides, specifically the part about mathematical operators).

3. Multiplication Table

- Write a script that displays the multiplication table for the number 5. The output should be in the format `5 x 1 = 5`, `5 x 2 = 10`, and so on up to `5 x 10 = 50`.

4. Population Growth:

- A country's population in 2000 was 50 million. If the population grows by 1 million every year, use `range(2000, 2011)` to print the population for each year from 2000 to 2010. The output should display the year followed by the population for that year. Now consider the case where the population grows each year by 2%

4. Nested Loops: Patterns

- Write a script that uses nested `for` loops to print the following pattern:

```
*
**
***
****
*****
```

5. Export Partners:

- You have a list of countries: ["USA", "Canada", "Mexico"]. Write a loop to print potential exporting partnerships by pairing each country with every other country in the list.

Outputs of a For Loop - Temporary or Lasting

- The output of a for loop can be temporary or lasting
 - Temporary: the output is not stored anywhere
 - Already covered this case in previous examples
 - Lasting: the output is stored in a variable

Outputs of a For Loop - Lasting Output Common Patterns

- Very often you use one of these two patterns:
 1. **Var creation**: create a dedicated variable before the loop and then update it in the loop
 2. **Var updating**: use a variable already existing and update it in the loop
 - Possibly updating (overwriting) the iterable itself
 - You do not need to create new variables
 - But **you overwrite the previous value** of the variable
 - Cannot do checks
 - If you did a mistake in the loop you will need to re-create the iterable:
Not convenient when developing code

For Loops Outputting a Scalar Data-Type - Var Creation

```
In [27]: print(f"Simple for loop example to sum integers in a list \n")
# define a list
simple_sequence = [1, 2, 3, 4, 5]
# define the var to store the output
simple_total = 0
# loop over the list
for val in simple_sequence:
    # add the current value to the running total
    simple_total += val
    # print the loop total
    print(f"The loop total is: {simple_total}")
# print the total
print(f"Total sum of integers in the list: {simple_total}\n")
```

Simple for loop example to sum integers in a list

```
The loop total is: 1
The loop total is: 3
The loop total is: 6
The loop total is: 10
The loop total is: 15
Total sum of integers in the list: 15
```

For Loops Outputting a Composite Data-Type - Var Creation

- A composite data-type main feature is its length
- Whether you know or do not know the length of the output determines what to do:
 1. **Known Output Length:** Pre-allocate a list of the known length and fill it using the loop
 2. **Unknown Output Length:** Pre-allocate an empty list and use the `extend()` or `append()` methods in the loop

For Loops Outputting a Composite Data-Type - Var Creation

Known Output Length - Pre-allocate

- Example: Computing the cumulative sum (cumsum) of a list of numbers
 - Each entry is computed as the sum of all previous elements
- The steps of this example are:
 1. Initialize a list of length equal to the original list to store the output
 2. Loop through the list using `range` and `len` to retrieve the current element position and, at each iteration:
 - A. Add the current element of the original list to the cumsum
 - B. Store the cumsum in the output list at the position of the current element

For Loops Outputting a Composite Data-Type - Var Creation

```
In [28]: #Computing cumulative sum with a pre-defined list
numbers = [1, 2, 3, 4, 5]
cumsum_fixed = [0] * len(numbers) #pre-allocate a list of zeros with the same
cumulative_sum = 0 # initialize cumulative sum to zero
for index in range(len(numbers)):
    cumulative_sum += numbers[index] # add the current number to the cumulat
    cumsum_fixed[index] = cumulative_sum # store the cumulative sum
print(f"Cumulative Sum with Pre-defined Length: {cumsum_fixed}") # print the
```

Cumulative Sum with Pre-defined Length: [1, 3, 6, 10, 15]

For Loops Outputting a Composite Data-Type - Var Creation

Unknown Output Length - Use extend/append

- Example: Create a list with the elements of multiple lists characterized by random length
- The steps of this example are:
 1. Initialize an empty list to store the output
 2. Perform multiple iterations and, at each iteration:
 - A. Randomly draw a number in [1, 3]
 - B. Create a list_loop with length equal to the drawn number
 - C. Extend the output list with the loop list

For Loops Outputting a Composite Data-Type - Var Creation

```
In [29]: import random # import the random package
randomLen_outputList = [] # create an empty list to store results
for iter_loop in range(1, 6): # does 5 loops in [1, 5]
    # takes a random draw of a number in [1, 3] with equal probability
    randomLen_loopDraw = random.randint(1, 3)
    # creates a list with length equal to randomLen_loop whose values are it
    randomLen_loopList = [iter_loop]*randomLen_loopDraw
    # extend the output list with the elements of the loop list
    randomLen_outputList.extend(randomLen_loopList)
    # print the length at each loop
    print(f"The number of elements of iteration {iter_loop} is: {randomLen_1
print(f"The whole 'extended' list is {randomLen_outputList}")#Print the whol
```

```
The number of elements of iteration 1 is: 1
The number of elements of iteration 2 is: 2
The number of elements of iteration 3 is: 2
The number of elements of iteration 4 is: 3
The number of elements of iteration 5 is: 1
The whole 'extended' list is [1, 2, 2, 3, 3, 4, 4, 4, 5]
```

```
In [30]: random.randint?
```

For Loops Outputting a Composite Data-Type - Var Creation (Nested Loop for Multiple Draws)

To better visualize the random nature of the script, here we replicate our code 3 times using an external loop

```
In [31]: for outer_loop in range(1,4): # does 3 loops
    randomLen_outputList = [] #pre allocate an empty list to store the exter
    for iter_loop in range(1, 6): # does 5 loops in [1, 5]
        randomLen_loopDraw = random.randint(1, 3) # will take a number in [1
        #creates a list with length equal to randomLen_loop whose values ar
        randomLen_loopList = [iter_loop]*randomLen_loopDraw
        # extend the output list with the elemnts of the loop list
        randomLen_outputList.extend(randomLen_loopList)
    #Print the whole output
    print(f"The whole 'extended' list of the external iteration {outer_loop}
```

The whole 'extended' list of the external iteration 1 is [1, 1, 1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5]

The whole 'extended' list of the external iteration 2 is [1, 2, 2, 3, 3, 4, 4, 5]

The whole 'extended' list of the external iteration 3 is [1, 1, 2, 2, 2, 3, 4, 4, 4, 5]

For Loops Outputting a Composite Data-Type - Var Updating

- There are cases where you will no longer need the original list
 - You can modify elements based on their index using `range` and `len`

```
In [32]: fruits = ['apple', 'banana', 'cherry'] # define a list
print(f"\nDefining original list as: {fruits}") # print the original list
print(f"Modifying list elements based on their index using range and len") #
for index in range(len(fruits)): # loop over the indices of the list
    element = fruits[index] # get the element at the current index
    fruits[index] = fruits[index].upper() # modify the element at the current index
    print(f"Element {element} is at index {index} - The modified element is: {fruits[index]}")
print(f"Modified List : {fruits}") # print the modified list
```

Defining original list as: ['apple', 'banana', 'cherry']

Modifying list elements based on their index using range and len

Element apple is at index 0 - The modified element is: APPLE

Element banana is at index 1 - The modified element is: BANANA

Element cherry is at index 2 - The modified element is: CHERRY

Modified List : ['APPLE', 'BANANA', 'CHERRY']

For Loops Outputting a Composite Data-Type - Var Updating

Access and modify elements based on their index using `enumerate`

```
In [33]: fruits = ['apple', 'banana', 'cherry'] # defining original list
print(f"\nRe-defining original list as: {fruits}") # printing original list
print(f"Modifying list elements based on their index using enumerate") # print
# using enumerate to get both index and element as variables to be used in the loop
for index, element in enumerate(fruits):
    fruits[index] = element.upper() # modifying the element at the index position
    # printing the modified element
    print(f"Element {element} is at index {index} - The modified element is: {fruits[index]}")
print(f"Modified List: {fruits}") # printing the modified list
```

Re-defining original list as: ['apple', 'banana', 'cherry']

Modifying list elements based on their index using enumerate

Element apple is at index 0 - The modified element is: APPLE

Element banana is at index 1 - The modified element is: BANANA

Element cherry is at index 2 - The modified element is: CHERRY

Modified List: ['APPLE', 'BANANA', 'CHERRY']

Exercises: For loops with Scalar and Composite Outputs

1. Simple Interest Calculation

- Given a principal amount, rate of interest, and number of years, calculate and the interest payment of each year using a for loop (the interest accruing each years is added to invested amount considered the next year).
- Input: `principal = 1000` , `rate = 5` , `years = 3`
- Now augment the previous loop so to store the interest of each year

2. Compound Interest Calculation

- Given a principal amount, rate of interest (compounded bi-annually), and number of years, calculate and print the compound interest for each year using a for loop.
- Input: `principal = 1000` , `rate = 5` , `years = 3` , `compounding_frequency = 2`
- Now augment the previous loop so to store the interest of each year

3. GDP Growth

- Given a list of GDP values for consecutive years, calculate the annual growth rate, print it and store it into a new list `gdp_growth` (pay attention to the length of `gdp_growth`)
- Input: `gdp_values = [2.5, 2.7, 2.9, 3.1]`

4. Stock Market Returns

- Given a list of stock prices for consecutive days, calculate the daily return (in percentage), print it and store it using the `.append` method
- Input: `stock_prices = [100, 101, 99, 102, 98]`

5. Trade Balance Calculation

- Given lists of export and import values for consecutive months, calculate the trade balance for each month.
- Input: `exports = [50, 55, 53, 52]` , `imports = [48, 54, 52, 53]`

6. Economic Indicators Dashboard

- Given lists of various economic indicators for consecutive months, create a summary that shows the average value of each indicator over the given period.
- Input: `gdp = [2.5, 2.6, 2.7]` , `inflation = [2.0, 2.1, 2.2]` , `unemployment = [5.0, 5.1, 5.2]`

Using Indexes to Access Related Elements of Multiple Lists

- Multiple lists might be used to store multiple information on the same observations
 - E.g., We can store information on the weight and height in two (ordered) lists corresponding to an (ordered) list of people
- We can use indexes to access elements of multiple lists
 - Very flexible as it can also combine elements of file structures other than lists

- E.g., You also store in each row of a dataframe (table) the info on daily calorie intake of the same people

Using Indexes to Access Related Elements of Multiple Lists

Calculate the cost of debt for each country using 3 lists reporting: country id, debt amounts and interest rates

```
In [34]: countries = ['USA', 'UK', 'Germany'] # list of countries
debt_amount = [25000, 20000, 15000] # list of debt amounts in million USD
interest_rate = [3, 2, 1] # list of interest rates
```

Using Indexes to Access Related Elements of Multiple Lists

Calculate the cost of debt using 3 lists of countries, debt amounts and interest rates

```
In [35]: countries = ['USA', 'UK', 'Germany'] # list of countries
debt_amount = [25000, 20000, 15000] # list of debt amounts in million USD
interest_rate = [3, 2, 1] # list of interest rates
print(f""""We have three lists: countries {countries}, debt_amount {debt_amount}
We will iterate through these lists using an index-based for loop""")
for i in range(len(countries)): # Simple For Loop with Index to analyze the
    country = countries[i] # Extract country using index
    debt = debt_amount[i] # Extract debt using index
    rate = interest_rate[i] # Extract rate using index
    yearly_cost = debt * (rate / 100) # Calculate yearly cost of debt
    print(f"Yearly cost of debt for {country}: ${yearly_cost:.2f} million")
```

We have three lists: countries ['USA', 'UK', 'Germany'], debt_amount [25000, 20000, 15000], and interest_rate [3, 2, 1].

We will iterate through these lists using an index-based for loop

Yearly cost of debt for USA: \$750.00 million

Yearly cost of debt for UK: \$400.00 million

Yearly cost of debt for Germany: \$150.00 million

The `zip` Function

- The `zip` function allows to combine multiple iterables element-wise
 - Outputs tuples that can be easily used in iteration
 - This is more efficient and readable than using indexes
 - Only works with iterables of the same length (it stops when the shortest iterable is exhausted)
 - It accepts lists, tuples, dictionaries, sets,

Using `zip` to Access Related Elements of Multiple Lists

```
In [36]: countries = ['USA', 'UK', 'Germany'] # list of countries
debt_amount = [25000, 20000, 15000] # list of debt amounts in million USD
```

```

interest_rate = [3, 2, 1] # list of interest rates
print(f"We have three lists: countries {countries}, debt_amount {debt_amount}
# Zipping the lists
financial_data = list(zip(countries, debt_amount, interest_rate))
print(f"The zip output is a list of tuples: {financial_data}\n")
for country, debt, rate in financial_data: # Simple For Loop to analyze the c
    # Calculate yearly cost of debt
    yearly_cost = debt * (rate / 100)
    print(f"Yearly cost of debt for {country}: ${yearly_cost:.2f} million")

```

We have three lists: countries ['USA', 'UK', 'Germany'], debt_amount [25000, 20000, 15000], and interest_rate [3, 2, 1]

The zip output is a list of tuples: [('USA', 25000, 3), ('UK', 20000, 2), ('Germany', 15000, 1)]

Yearly cost of debt for USA: \$750.00 million

Yearly cost of debt for UK: \$400.00 million

Yearly cost of debt for Germany: \$150.00 million

🏆 Exercises: For loops with multiple lists

1. Energy Consumption and Dominant Source

- Given lists of countries, their oil, coal, and natural gas consumption (in million BTUs), calculate the total energy consumption for each country.

```

countries = ['USA', 'China', 'Russia']
oil_consumption = [19000, 13000, 3000]
coal_consumption = [14000, 19000, 5500]
gas_consumption = [30000, 7500, 17000]

```

For Loops and Control Flow

- The body of a for loop might be used on values that we want to treat differently
 - If you were writing the code by hand you would write different code for different cases
 - Using iteration we need some tools to allow for different code to be executed in different cases

For Loops and Control Flow

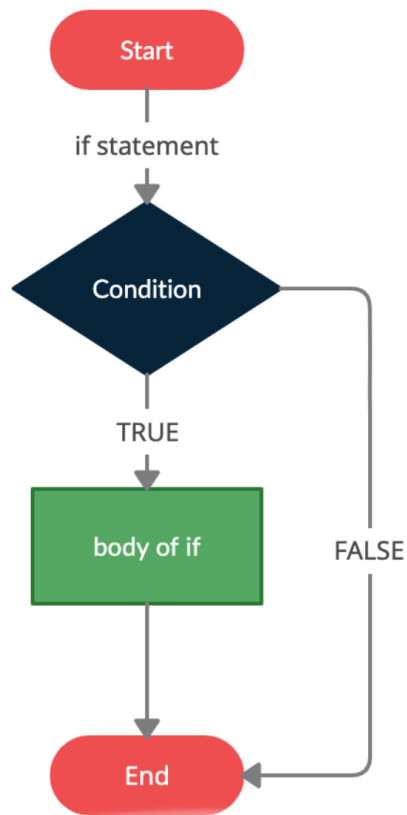
- Control flow statements** allow you to control the flow of the program based on conditions
 - if** statement: Executes the indented body if the condition is true
 - An **if** statement can be **optionally** followed by one or more **elif** blocks and a catchall **else** block at the end
 - elif** statement: Executes the indented body if the previous **if** or **elif** condition is false and the condition of the current **elif** is true

- `else` statement: Executes the indented body if all previous conditions are false

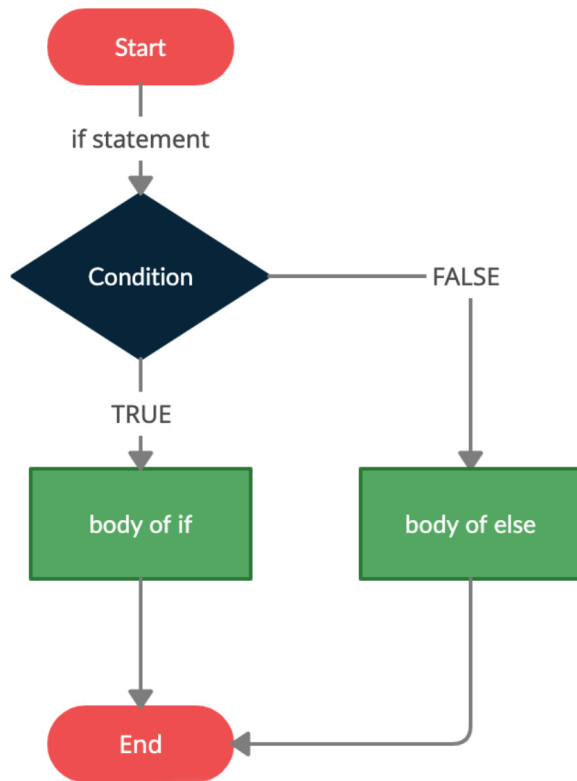
For Loops and Control Flow

- It is legitimate to write an `if` conditional without an `else`
 - But best to **also** have an `else` so that you have to think of the alternative case and not miss relevant values

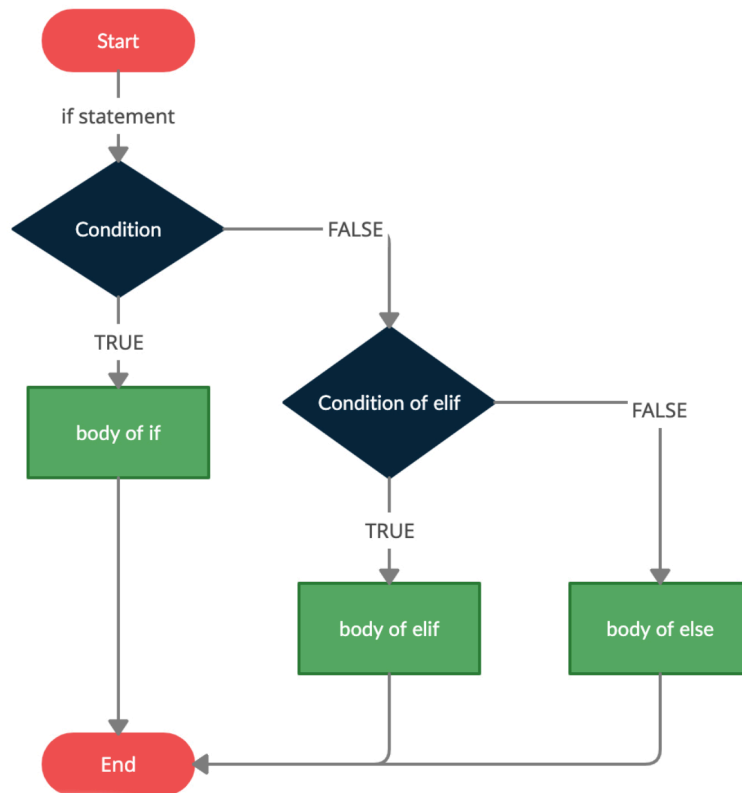
`if`



`if, else`



if, elif, else



For Loops and Control Flow - An `if` Example

```
In [37]: sequence = [0, 1, None, "Ciao", 4, None, 6, 7]
print(f"\nOriginal sequence: {sequence}")
# Using if
print("\nLoop using if to print only numeric values:")
for item in sequence:
    if type(item) in [int, float]:
        print(f"Loop item = {item}")
```

Original sequence: [0, 1, None, 'Ciao', 4, None, 6, 7]

Loop using if to print only numeric values:

Loop item = 0

Loop item = 1

Loop item = 4

Loop item = 6

Loop item = 7

For Loops and Control Flow - An `if`, `elif`, `else` Example

```
In [38]: sequence = [0, 1, None, "Ciao", 4]
print(f"\nOriginal sequence: {sequence}")
print("\nLoop using if, elif, and else to print different messages based on")
for item in sequence:
    if type(item) in [int, float]:
        print(f"We are in the if chunk since the loop value type of {item} is numeric")
    elif type(item) in [str]:
        print(f"We are in the elif chunk since the value type of {item} is string")
    else:
        print(f"We are in the else chunk since the value type of {item} is neither numeric nor string")
```

Original sequence: [0, 1, None, 'Ciao', 4]

Loop using if, elif, and else to print different messages based on the type of the loop value

We are in the if chunk since the loop value type of 0 is numeric

We are in the if chunk since the loop value type of 1 is numeric

We are in the else chunk since the value type of None is neither numeric nor string

We are in the elif chunk since the value type of Ciao is string

We are in the if chunk since the loop value type of 4 is numeric

Group Operations

- Group operations are used to perform operations on subsets of data
 - E.g., Calculating the average income (operation) of individuals (observations) in different countries (groups)
- Group operations are a fundamental tool of data analysis
 - It is the most immediate way to identify patterns in data

- It is often important to know if one variable increases/decreases with another

Group Operations: Multiple Lists and Control Flow Statements

- We can use iteration, multiple lists and control flow to perform group operations
 - It is not the simplest (or most efficient) way, but it is the most flexible
 - It was the only way to do group operations before the advent of more modern tools (e.g., pandas)
- Pandas performs complex group operations with few lines of code
 - Just convenience wrappers of for loops and control flow statements
 - Hard to understand if you are not familiar with the underlying logic
 - Studying for loops allow an easier understanding

Group Operations Example: Computing Average Income by Country

```
In [39]: # Lists of individuals' income and country
incomes = [50000, 52000, 70000, 68000, 71000, 45000, 43000, 46000]
countries = ["CountryA", "CountryA", "CountryB", "CountryB", "CountryB", "Co
```

For each country we will need to initialize

- A variable to store the number of GDP observations by country to compute the average
- A variable to store the values of the computed GDP averages

Initialize the Output

```
In [40]: # Lists of individuals' income and of their countries
incomes = [50000, 52000, 70000, 68000, 71000, 45000, 43000, 46000]
countries = ["CountryA", "CountryA", "CountryB", "CountryB", "CountryB", "Co

# Initializing variables for total income and count for each country
total_income_a = 0
count_a = 0
total_income_b = 0
count_b = 0
total_income_c = 0
count_c = 0
```

Perform the for Loop to Compute the Total Income and Number of Individuals by Country

```
In [41]: for i in range(len(countries)): # Iterating over each country and income
        country = countries[i]
        income = incomes[i]
        # Conditional logic to update the total income and count for each countr
```

```

if country == "CountryA":
    total_income_a += income # Adding income to CountryA's total
    count_a += 1             # Incrementing CountryA's count
elif country == "CountryB":
    total_income_b += income # Adding income to CountryB's total
    count_b += 1             # Incrementing CountryB's count
elif country == "CountryC":
    total_income_c += income # Adding income to CountryC's total
    count_c += 1             # Incrementing CountryC's count
print(f"total_income_a: {total_income_a}, count_a: {count_a}")
print(f"total_income_b: {total_income_b}, count_b: {count_b}")
print(f"total_income_c: {total_income_c}, count_c: {count_c}")

```

```

total_income_a: 102000, count_a: 2
total_income_b: 209000, count_b: 3
total_income_c: 134000, count_c: 3

```

Computing Average Income by Country: Computing and Printing Averages

```

In [42]: # Calculating average incomes for each country
# Using conditional to avoid division by zero
average_income_a = total_income_a / count_a
average_income_b = total_income_b / count_b
average_income_c = total_income_c / count_c

# Printing average incomes
print(f"Average Income in CountryA: {average_income_a:.2f}")
print(f"Average Income in CountryB: {average_income_b:.2f}")
print(f"Average Income in CountryC: {average_income_c:.2f}\n")

```

```

Average Income in CountryA: 51000.00
Average Income in CountryB: 69666.67
Average Income in CountryC: 44666.67

```

Computing Average Income by Country: The Whole Code

```

In [43]: ### Compute and Print Average Income by Country

# Lists of individuals' income and of their countries
incomes = [50000, 52000, 70000, 68000, 71000, 45000, 43000, 46000]
countries = ["CountryA", "CountryA", "CountryB", "CountryB", "CountryB", "Co

# Initializing variables for total income and count for each country
total_income_a = 0
count_a = 0
total_income_b = 0
count_b = 0
total_income_c = 0
count_c = 0

# Iterating over each country and income
for i in range(len(countries)):
    country = countries[i]

```

```

income = incomes[i]

# Conditional logic to update the total income and count for each country
if country == "CountryA":
    total_income_a += income # Adding income to CountryA's total
    count_a += 1 # Incrementing CountryA's count
elif country == "CountryB":
    total_income_b += income # Adding income to CountryB's total
    count_b += 1 # Incrementing CountryB's count
elif country == "CountryC":
    total_income_c += income # Adding income to CountryC's total
    count_c += 1 # Incrementing CountryC's count

# Calculating average incomes for each country
# Using conditional to avoid division by zero
average_income_a = total_income_a / count_a if count_a else None # 'if' added
average_income_b = total_income_b / count_b if count_b else None
average_income_c = total_income_c / count_c if count_c else None

# Printing average incomes
print(f"Average Income in CountryA: {average_income_a}")
print(f"Average Income in CountryB: {average_income_b}")
print(f"Average Income in CountryC: {average_income_c}\n")

```

```

Average Income in CountryA: 51000.0
Average Income in CountryB: 69666.66666666667
Average Income in CountryC: 44666.666666666664

```

How can we compute the % of observations in the list belonging to each country?

In []:

Computing Average Income by Country: A More Powerful Approach to this Problem

- This approach allowed us to avoid identify by hand all the elements belonging to each country to do computations
 - But it still required us to define one (or more) output variable for each group
 - In real-world data it might be unfeasible to perform it manually
- Need to make the code able to **identify** and **store** output for an arbitrarily large number of groups (countries)
 - We will see that sets are the missing tool to do this job more efficiently



Exercises: For loops with Control Flow - Group Operations

1. Use the provided lists of product categories and sales amounts to calculate the total sales for each product category.
Directions: Create variables for each category's total sales and iterate through the

lists updating the relevant sale totals. Finally, print the total sales for "Electronics", "Clothing", and "Groceries".

```
categories = ["Electronics", "Clothing", "Groceries",  
             "Electronics", "Clothing", "Groceries", "Electronics",  
             "Clothing", "Groceries", "Electronics"]  
sales = [200, 150, 50, 300, 120, 80, 250, 160, 60, 220]
```

2. Use the provided lists for product names and their ratings to classify products based on their rating.

Directions: Create separate lists for each rating category, and as you iterate through the products and their ratings, append each product to the appropriate list. Finally, print the names of the products in each rating category.

```
products = ["Laptop", "Shirt", "Bread", "Smartphone", "Jeans",  
           "Milk", "Headphones", "Dress", "Eggs", "Tablet", "Coat",  
           "Cheese", "Speakers", "Skirt", "Butter"]  
ratings = [5, 3, 4, 4, 2, 5, 3, 4, 1, 5, 3, 4, 5, 2, 3]
```

Loop Control Flow Statements

- Allow you to control the iteration of the loop(s)
- There are various commands, but we will briefly look at:
 - `continue` : Skips the current iteration and moves to the next
 - `break` : Exits the loop altogether

Loop Control Flow Statements - `continue`

`continue` : Allows you to skip the current iteration value and moving to the next

```
In [44]: sequence = [0, 1, None, "Ciao", 4, None, 6, 7]  
print(f"The list is: {sequence}")  
print("\nLoop using 'continue' when encountering non-numeric values:")  
for item in sequence:  
    if type(item) not in [int, float]:  
        continue  
    print(f"continue: item = {item}")
```

The list is: [0, 1, None, 'Ciao', 4, None, 6, 7]

```
Loop using 'continue' when encountering non-numeric values:  
continue: item = 0  
continue: item = 1  
continue: item = 4  
continue: item = 6  
continue: item = 7
```

Loop Control Flow Statements - `break`

`break` : Exits the loop altogether

```
In [45]: print(f"The list is: {sequence}")
print("\nLoop using 'break' when encountering string values:")
for item in sequence:
    if type(item) is str:
        break
    print(f"break: item = {item}")
```

The list is: [0, 1, None, 'Ciao', 4, None, 6, 7]

Loop using 'break' when encountering string values:

break: item = 0

break: item = 1

break: item = None

Understanding Break and Continue in Nested Loops - Continue in Inner Loop

`continue` in **inner** loop skips current iteration of **inner** loop and continues with next iteration of **inner** loop

```
In [46]: print("Example 1: Using 'continue' in inner loop")
for i in range(3):
    for j in range(3):
        # if j=1 the code goes to next inner iteration (and skips printing)
        if j == 1:
            continue
        print(f"i: {i}, j: {j}")
```

Example 1: Using 'continue' in inner loop

i: 0, j: 0

i: 0, j: 2

i: 1, j: 0

i: 1, j: 2

i: 2, j: 0

i: 2, j: 2

Understanding Break and Continue in Nested Loops - Break in Inner Loop

`break` in **inner** loop exits the innermost loop and starts the next iteration of the **outer** loop

```
In [47]: print("Example 2: Using 'break' in inner loop")
for i in range(3):
    for j in range(3):
        if j == 1:
            break # if j=1 the code breaks the inner loop and goes to next c
        print(f"i: {i}, j: {j}")
```

Example 2: Using 'break' in inner loop

i: 0, j: 0

i: 1, j: 0

i: 2, j: 0

Understanding Break and Continue in Nested Loops - Continue in Outer Loop

`continue` in **outer** loop skips the current iteration of **outer** loop and continues with next iteration of **outer** loop

```
In [48]: # Using 'continue' in outer loop
print("Example 3: Using 'continue' in outer loop")
for i in range(3):
    if i == 1:
        continue # if i=1 the code goes to next outer iteration
    for j in range(3):
        print(f"i: {i}, j: {j}")
```

Example 3: Using 'continue' in outer loop

i: 0, j: 0

i: 0, j: 1

i: 0, j: 2

i: 2, j: 0

i: 2, j: 1

i: 2, j: 2

Understanding Break and Continue in Nested Loops - Break in Outer Loop

`break` in **outer** loop breaks the outer loop altogether

```
In [49]: # Using 'break' in outer loop
print("Example 4: Using 'break' in outer loop")
for i in range(3):
    if i == 1:
        break # if i=1 the code breaks the outer loop
    for j in range(3):
        print(f"i: {i}, j: {j}")
```

Example 4: Using 'break' in outer loop

i: 0, j: 0

i: 0, j: 1

i: 0, j: 2

🏆 Exercises: For loops with Control Flow

1. Budget Performance Analysis

- Given lists of budgeted and actual expenses for consecutive months, determine the performance against the budget. If the actual expenses exceed the budget by more than 50 in any month, print a warning message. If the actual expenses are exactly as

budgeted, print a neutral message. If the actual expenses are below the budget by any amount, print a positive message.

- Input:
 budgeted_expenses = [1000, 1100, 1050, 1075]
 actual_expenses = [1050, 1120, 1300, 1075]
- Hint:
 1. Loop through the budgeted and actual expenses.
 2. Compare the actual expenses with the budgeted expenses for each month and print the appropriate message.

2. Trade Surplus Analysis

Given lists representing monthly trade balances (exports minus imports) for different countries, determine the first country that has a trade surplus (positive balance) for every month in the year. Once found, print the country's name and stop the analysis.

Input:

```
CountryB_balances = [-10, -20, 30, 40, -50, 60, -70, 80, -90, 100, -110, 120]
CountryA_balances = [50, 60, 70, 80, 90, 100, 110, 120, 130, 140, 150, 160]
CountryC_balances = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120]
```

3. Monthly Sales Target Achievement

Given lists representing monthly sales figures for different sales representatives, identify and print the names of representatives who have achieved or exceeded the sales target for at least 10 months in the year. If a representative doesn't meet this criterion, skip to the next one.

Input:

```
RepJohn_sales = [5000, 6000, 7000, 8000, 9000, 10000, 11000, 12000, 13000, 14000, 15000, 16000]
RepAnne_sales = [12000, 10200, 15000, 11500, 12000, 12500, 11000, 13500, 14500, 14500, 15000, 15500]
RepJane_sales = [4000, 3500, 3000, 4500, 5000, 6000, 5500, 5800, 5900, 6000, 6100, 6200]
RepDoe_sales = [10000, 10500, 11000, 11500, 12000, 12500, 13000, 13500, 14000, 14500, 15000, 15500]
sales_target = 10000
```

List Comprehensions in Python

- List comprehensions provide a concise way to create lists using for/if statements
 - Basically a shorthand of for-if loops
 - Very common since they make code more readable

- Allow selecting and transforming elements that meet a specific condition
- No need to pre-allocate the output variable, it is created automatically

List Comprehensions in Python

- Basic Syntax:

```
result_var = [code_doing_stuff_on_value for value in iterable if condition]
```

- This is a more compact and readable alternative to using a for loop with conditions

```
result_var = []
for value in iterable:
    if condition:
        result.append(code_doing_stuff_on_value)
```

Example of Traditional For Loop vs. List Comprehension - Square of Even Numbers from 0 to 9

- For Loop:

```
In [50]: squared_numbersLoop = []
        for x in range(10):
            if x % 2 == 0:
                squared_numbersLoop.append(x**2)
        print(f"List of squared numbers using a for loop: {squared_numbersLoop}")
```

List of squared numbers using a for loop: [0, 4, 16, 36, 64]

- Equivalent list comprehension:

```
In [51]: squared_numbersLC = [x**2 for x in range(10) if x % 2 == 0]
        print(f"List of squared numbers using list comprehension: {squared_numbersLC}")
```

List of squared numbers using list comprehension: [0, 4, 16, 36, 64]

🏆 Exercise on Data Types, for Loops and Conditionals - DO TOGETHER?

You are provided with a list of student exam scores, including some missing values (`None`) and special scores labeled as `"30L"` (representing 30 with distinction. Your task is to analyze this data with the following objectives:

1. Count how many times the score `"30L"` appears in the data.
2. Calculate the percentage of missing observations (`None` in the list).
3. Compute the average score of the grades, treating `"30L"` as equal to `33` and excluding missing values (`None`). **Hint: Pay attention to data types!**

4. Compute the average score again, but this time impute the missing values with the average grade, which is `26`.
5. Redo the exercise using list comprehensions for point 3 and 4

```
student_scores = ["28", "27", "25", "27", None, None, "29", "28",  
None, "30L", "28",  
"30", "30", "30L", "25", "28", "29", "30L", "29", "28", "30",  
"25", "28", "30L", "29", "30L", None, "30L", "25", "30", "30",  
"28", "30", "28", "30L", "25", "28", "29", "30L", "25", "30L",  
None, "30L", "29", "30", "30L", "27", "30", "28", "30L"]
```

Exercises on List Comprehension

1. Create a list of even numbers from 1 to 20 that are also divisible by 3.
2. Generate a list that contains each odd number from 1 to 15 multiplied by 2 and then subtracted from its square.
3. Given the following list of words, create a new list containing the length of each word divided by 2, but only for words that are longer than 5 characters.

```
words = ["environment", "data", "analysis", "python",  
"programming", "science", "visualization"]
```

4. Given the following list of numbers, create a list where each number is replaced with 'High' if the number is greater than 50, 'Medium' if the number is between 25 and 50, and 'Low' if the number is less than 25.

```
numbers = [12, 55, 34, 22, 75, 43, 56, 28, 19, 66, 37, 48, 91,  
5, 33]
```

While Loops

- Basic syntax

```
while condition:  
    body
```

- A loop that defines a body of code to be executed as long as a condition is true
 - Most useful when you do not know how many times you need to iterate
 - The loop can also be explicitly ended with a `break`
- For each `for` loop there is an equivalent `while` loop
- We won't see `while` loops in depth but sometimes are very useful

While Loops - Example: Savings Growth

- **Goal:** Given an initial balance, a target balance and the interest rate, calculate the number of years it takes to reach the target balance
- **Condition:** The current balance is below the target balance
- **Body of the loop:** For each year:
 1. Compute the current balance based on the one of the previous and the interest rate
 2. Increment the year counter
 3. Print info on the balance of the given year

While Loops - Example: Savings Growth

```
In [52]: # Example: Calculating Time to Reach Savings Target
initial_balance = 1000 # Initial savings balance in USD
target_balance = 2000 # Target savings balance in USD
annual_interest_rate = 15 # Annual interest rate in percentage
```

While Loops - Example: Savings Growth

```
In [53]: initial_balance = 1000 # Initial savings balance in USD
target_balance = 2000 # Target savings balance in USD
annual_interest_rate = 15 # Annual interest rate in percentage
years = 0 # Counter for years
print(f"Calculating time to reach a savings target of ${target_balance} with")
while initial_balance < target_balance: # While Loop to calculate the time t
    annual_interest = initial_balance * (annual_interest_rate / 100) # Calcula
    initial_balance += annual_interest # Update the current balance
    years += 1 # Increment the year counter
    print(f"After {years} years, the balance is: ${initial_balance:.2f}")
# Print the total number of years needed to reach the target balance
print(f"It will take {years} years to reach a balance of ${target_balance}.")
```

Calculating time to reach a savings target of \$2000 with an initial balance of \$1000 and an annual interest rate of 15%

After 1 years, the balance is: \$1150.00
 After 2 years, the balance is: \$1322.50
 After 3 years, the balance is: \$1520.88
 After 4 years, the balance is: \$1749.01
 After 5 years, the balance is: \$2011.36
 It will take 5 years to reach a balance of \$2000.

Tuples

Tuples

- A compound data type used to safely store a **fixed number** of values
 - Leverages on the fact that tuples are immutable
 - Frequent use-cases

- Used as iterable - That's what `zip` and `enumerate` do
- **Return values within function definitions** (avoiding unwanted side-effects)
- Create keys of dictionaries

Tuples Syntax Rules

A tuple can be defined with or without parentheses:

`( el1, ..., eln, ... )` or `el1, ..., eln, ...`

except the empty tuple which always requires parentheses:

`( )`

- Elements must always be separated using commas
 - Otherwise they are interpreted as strings to concatenate

```
In [54]: # tuple with parentheses
p_tuple = ( 'physics', 'chemistry', 1997, 2000 )
p_tuple
```

```
Out[54]: ('physics', 'chemistry', 1997, 2000)
```

```
In [55]: # tuple with no parentheses, as when using enumerate/zip in iteration to extend
np_tuple = 'physics', 'chemistry', 1997, 2000
np_tuple
```

```
Out[55]: ('physics', 'chemistry', 1997, 2000)
```

```
In [56]: # the empty tuple, always with parentheses
e_tuple = ()
type(e_tuple)
```

```
Out[56]: tuple
```

```
In [57]: # a tuple and a string
a_tuple = 'chemistry', 'physics'
a_string = 'chemistry' 'physics'
print(f"a_tuple = 'chemistry', 'physics' results in: {a_tuple}")
print(f"a_string = 'chemistry' 'physics' results in: {a_string}")
```

```
a_tuple = 'chemistry', 'physics' results in: ('chemistry', 'physics')
a_string = 'chemistry' 'physics' results in: chemistryphysics
```

Comparing Tuples and Lists

- Similarities

- Tuples are ordered, like lists. Their elements are ordered by indexes, starting with 0.
- Therefore tuples can be
 - indexed, sliced with/without strides,
 - concatenated,
 - iterated over

Comparing Tuples and Lists

- Differences
 - Tuples are immutable
 - **The length of the tuple is fixed**
 - Neither indexing, slicing nor concatenation can update tuples
 - If the elements of a tuple are mutables (e.g., lists), these can change

A Operations, Methods, Functions and Iterations with Tuples

Expression with <code>t = (3, 6, 4)</code>	Results	Description
<code>len(t)</code>	3	length
<code>max(t)</code>	6	max if of the same type
<code>min(t)</code>	3	min if of the same type
<code>t + (7, 9)</code>	<code>(3, 6, 4, 7, 9)</code>	concatenation
<code>t * 2</code>	<code>(3, 6, 4, 3, 6, 4)</code>	repetition
<code>3 in t</code>	True	membership
<code>for x in t: print(x, end="")</code>	3 6 4	iteration via elements
<code>for i in range(len(t)): print(i, end="")</code>	0 1 2	iteration via indexes
<code>for i,x in enumerate(t): print((i,x), end="")</code>	<code>(0,3) (1,6) (2,4)</code>	iteration via indexes and elements
<code>tuple(sorted(t))</code>	<code>(3, 4, 6)</code>	sorted tuple

Unordered Types

- Unordered types `set` and `dict`
- Very useful in many applications

Sets

Sets - Unordered Mutable Collections of Unique Items

- Can be created using the `set()` function or curly brackets `{}`
- Most useful to check (and possibly remove) duplicates
 - To identify the unique elements of a collection
 - **Fast** test if an element is in a collection (`in`)
- Allow to perform set operations: union, intersection, difference, symmetric difference
 - See [here](#) for a complete list of methods

```
In [58]: st = {1, 2, 2, 'a', 'b'} # sets are unordered
print(f"The code st = {{1, 2, 2, 'a', 'b'}} results in the set: {st}")
```

The code `st = {1, 2, 2, 'a', 'b'}` results in the set: `{1, 2, 'a', 'b'}`

Identifying Unique Elements

- Sets can be very useful if you want to identify the unique elements of a collection
 - For example, if you want to identify the unique items in a list of items
 - **Supercharges iteration**

Computing Average Income by Country: A More Powerful Approach to the Problem

- Our previous approach required us to define one (or more) output variable for each group
 - In real-world data it might be unfeasible to perform it manually

```
In [59]: incomes = [89543, 76227, 89707, 95918, 58683, 44206, 76055, 54332, 86597, 56
countries = ['CountryP', 'CountryA', 'CountryV', 'CountryE', 'CountryO', 'Cc
```

Computing Average Income by Country: Using Sets and Lists

- Need to make the code able to deal with an arbitrary number of groups (countries)
- Using **sets** we can easily **identify the unique groups** in our data
- Using **lists** we can **store their outputs**
- We have now what we need to compute statistics on an arbitrarily large number of groups

Computing Average Income by Country: Using Sets and Lists

1. Identify and store the number of unique groups (countries) using `set()`
2. Define one or more lists with length equal to the number of groups to store output

3. Do computations on the original lists and store the result in the (intermediate) output lists
4. Do further computations on the output lists and print the final result

Computing Average Income by Country: Using Sets and Lists

```
In [60]: incomes = [89543, 76227, 89707, 95918, 58683, 44206, 76055, 54332, 86597, 56
countries = ['CountryP', 'CountryA', 'CountryV', 'CountryE', 'CountryO', 'Co
unique_countries = sorted(list(set(countries))) # Create a list of unique cou
average_incomes = [0]*len(unique_countries) # Initialize a list to store the
for idx, unique_country in enumerate(unique_countries): # Iterate over each u
    total_income = 0 # Initialize total income for the current country
    count = 0 # Initialize count for the current country
    # Iterate over each element of the lists country/incomes
    for country, income in zip(countries, incomes):
        if country == unique_country: # Check if the country in the list mat
            total_income += income # Add income to total for this unique co
            count += 1 # Increment count for this unique countr
    # Here we are back in the outermost loop! So we have already gathered al
    average_incomes[idx] = total_income / count # Compute and store the aver
print(f"Average Incomes of Countries \n{unique_countries}\nare:\n{average_ir
```

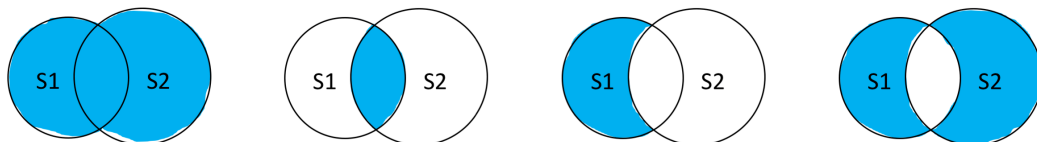
Average Incomes of Countries

['CountryA', 'CountryB', 'CountryC', 'CountryD', 'CountryE', 'CountryF', 'CountryG', 'CountryI', 'CountryJ', 'CountryK', 'CountryL', 'CountryM', 'CountryN', 'CountryO', 'CountryP', 'CountryQ', 'CountryR', 'CountryS', 'CountryT', 'CountryU', 'CountryV', 'CountryW']

are:

[54167.25, 87175.0, 60772.666666666664, 75655.5, 71287.5, 38207.0, 55272.25, 54650.333333333336, 72083.83333333333, 73177.125, 48698.0, 83863.0, 76668.33333333333, 52766.0, 85139.2, 66465.28571428571, 72754.25, 75799.66666666667, 72549.75, 69967.66666666667, 67569.77777777778, 60566.166666666664]

Set Methods



- `S1.union(S2)` , `S1|S2` — elements in S1 or S2, or both
- `S1.intersection(S2)` , `S1&S2` — elements in both S1 and S2
- `S1.difference(S2)` , `S1-S2` — elements in S1 but not in S2
 - The difference is not symmetric: `S1-S2` is not the same as `S2-S1`
- `S1.symmetric_difference(S2)` , `S1^S2` — elements in S1 or S2 but not both

Set Methods

```
In [61]: st1 = set([1, 2, 3])
st2 = set([3, 4, 5])

print(f"The union of {st1} and {st2} is {st1.union(st2)}")
print(f"The intersection of {st1} and {st2} is {st1.intersection(st2)}")
print(f"The difference of {st1} and {st2} is {st1.difference(st2)}")
print(f"The difference of {st2} and {st1} is {st2.difference(st1)}")
print(f"The symmetric difference of {st1} and {st2} is {st1.symmetric_differ
```

The union of {1, 2, 3} and {3, 4, 5} is {1, 2, 3, 4, 5}
The intersection of {1, 2, 3} and {3, 4, 5} is {3}
The difference of {1, 2, 3} and {3, 4, 5} is {1, 2}
The difference of {3, 4, 5} and {1, 2, 3} is {4, 5}
The symmetric difference of {1, 2, 3} and {3, 4, 5} is {1, 2, 4, 5}

Identifying Unique Elements and Membership Tests

- Sets can be very useful if you want to check for data consistency
 - At the very least we want to be know if there are typos!

Identify Typos using sets

```
In [62]: month_lst = ['Oct', 'Sep', 'Sep', 'Apr', 'Dec', 'Feb', 'May', 'Aug', 'Apr',
month_st = set(month_lst) # Create a set from the list
print(f"Let's assume to have a list with the birthday dates of 100 people: m
print(f"month_st = set(month_lst) results in a set of length {len(month_st)}
print(f"This is a quick and dirty way to spot typos")
```

Let's assume to have a list with the birthday dates of 100 people: month_lst = ['Oct', 'Sep', 'Sep', 'Apr', 'Dec']... (and more)

month_st = set(month_lst) results in a set of length 14:
['Apr', 'Aug', 'Dec', 'Decc', 'Feb', 'Jan', 'Jul', 'Jum', 'Jun', 'Mar', 'May', 'Nov', 'Oct', 'Sep']

This is a quick and dirty way to spot typos

Identify Typos using sets

If we know the values that a variable is supposed to assume we can do better

```
In [63]: # Create a set with expected month abbreviations
months_noTypo = {'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Se
# Check the difference between the two sets
print(f"Defining month abbreviations: {months_noTypo}")
print(f"Running month_st.difference(months_noTypo) allows to identify the ty
```

Defining month abbreviations: {'Apr', 'Nov', 'Jan', 'May', 'Feb', 'Dec', 'Sep', 'Jun', 'Jul', 'Mar', 'Aug', 'Oct'}
Running month_st.difference(months_noTypo) allows to identify the typos directly: {'Decc', 'Jum'}

- Running `month_st.difference(months_noTypo)` removes from the set of months the ones in the (expected) `months_correct`
 - The ones left are typos

Correct the Typos Using `for` and `if`

```
In [64]: # Now we can easily correct the typos
for i in range(len(month_lst)):
    if month_lst[i] == 'Decc':
        month_lst[i] = 'Dec'
    if month_lst[i] == 'Jum':
        month_lst[i] = 'Jun'

# Check that the typos have been corrected
print(f"After correcting 'Decc' to 'Dec' and 'Jum' to 'Jun' in month_lst, th
print(f"And the elements in month_lst are the ones in months_noTypo since:\r
```

After correcting 'Decc' to 'Dec' and 'Jum' to 'Jun' in month_lst, the unique elements of the list becomes:

```
['Apr', 'Aug', 'Dec', 'Feb', 'Jan', 'Jul', 'Jun', 'Mar', 'May', 'Nov', 'Oct', 'Sep']
```

And the elements in month_lst are the ones in months_noTypo since:
`set(month_lst) == months_noTypo` results in True

Exercise: Correct the Typos Using List Comprehensions

In []:

Dictionaries

Dictionaries

- Dictionaries do not use indexes, like lists. They use so-called **keys**, mapped to **values**.
- They are similar to a bilingual dictionary, e.g.,
 - From the English key 'one' to the German value 'ein',
 - From the English key 'two' to the German value 'zwei' etc.

Dictionary Syntax rules

The syntax of a dictionary is:

```
{ ..., key: value, .... }
```

- A `key` and its `value` are separated by a colon. They form an **item**.
- Each item `key: value` is separated commas

```
In [65]: # Create a dictionary
my_dict = {'one': 'ein', 'two': 'zwei', 'three': 'drei'}
# Display the dictionary
print(f"my_dict = {{'one': 'ein', 'two': 'zwei', 'three': 'drei'}} results in:
my_dict = {'one': 'ein', 'two': 'zwei', 'three': 'drei'} results in:
{'one': 'ein', 'two': 'zwei', 'three': 'drei'}
```

Rules for Keys and Values

- Keys
 - Each **key** is **unique** so as to identify its own **value**.
 - A key is **immutable** so:
 - It can be an atomic type (e.g., a Boolean, an integer) or a string
 - It cannot be a list or a dictionary (their length is not fixed, so mutable)
- Values
 - A **value** can be any type, including a list or a dictionary

Dictionary Key Restrictions

```
In [66]: my_dict = {
            'id' : 3232322,
            'school': 'Galilei',
            'courses': [ 'English', 'Maths' ],
            'students': { 'Sarah': 21, 'Bob': 19 }
          }
print(my_dict)
```

```
{'id': 3232322, 'school': 'Galilei', 'courses': ['English', 'Maths'], 'students': {'Sarah': 21, 'Bob': 19}}
```

```
In [67]: # keys must be immutable: lists are NOT accepted as keys
my_dict = {[ 'courses' , 'English', 'Maths' ]: { 'Sarah': 21, 'Bob': 19 } }
```

```
-----
TypeError                                 Traceback (most recent call last)
Cell In[67], line 2
      1 # keys must be immutable: lists are NOT accepted as keys
----> 2 my_dict = {[ 'courses' , 'English', 'Maths' ]: { 'Sarah': 21, 'Bob': 19 } }

TypeError: unhashable type: 'list'
```

Dictionary Key Restrictions

```
In [68]: # keys are immutable: dictionaries are NOT accepted as keys
my_dict = {
            { 'courses' : [ 'English', 'Maths' ] }: { 'Sarah': 21, 'Bob': 19 }
          }
```

```
print(my_dict)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[68], line 2
      1 # keys are immutable: dictionaries are NOT accepted as keys
----> 2 my_dict = {
      3     { 'courses' : [ 'English', 'Maths' ] }: { 'Sarah': 21, 'Bob':
19 }
      4     }
      6 print(my_dict)

TypeError: unhashable type: 'dict'
```

Dictionary Key Restrictions

```
In [69]: # keys are immutable: tuples are accepted as keys
my_dict = {
    ( 'courses' , 'English', 'Maths' ): { 'Sarah': 21, 'Bob': 19 }
}
print(my_dict)
```

```
{('courses', 'English', 'Maths'): {'Sarah': 21, 'Bob': 19}}
```

Dictionary Methods - A Simple Example

```
In [70]: # The rooms dictionary
rooms = {
    'Hall': {
        'south': 'Kitchen',
        'item': 'key',},
    'Kitchen': {
        'north': 'Hall',
        'item': 'monster'},
    'Dining Room': {
        'west': 'Hall',
        'item': 'potion'}}
print(f"Initial 'rooms' dictionary: \n{rooms}")
```

```
Initial 'rooms' dictionary:
{'Hall': {'south': 'Kitchen', 'item': 'key'}, 'Kitchen': {'north': 'Hall',
'item': 'monster'}, 'Dining Room': {'west': 'Hall', 'item': 'potion'}}
```

Dictionary Methods

- `D.keys()` that returns the keys of a dictionary
- `D.values()` that returns the values of a dictionary

```
In [71]: # Using D.keys() method
print(f"Keys of the 'rooms' dictionary are rooms.keys(): \n{rooms.keys()}\n")
```

```
# Using D.values() method
print(f"Values of the 'rooms' dictionary are rooms.values(): \n{rooms.values()}"
```

Keys of the 'rooms' dictionary are rooms.keys():
dict_keys(['Hall', 'Kitchen', 'Dining Room'])

Values of the 'rooms' dictionary are rooms.values():
dict_values([{'south': 'Kitchen', 'item': 'key'}, {'north': 'Hall', 'item': 'monster'}, {'west': 'Hall', 'item': 'potion'}])

Dictionary Methods

- `D.items()` that returns the key-value pairs of a dictionary
- `D.get(key)` that returns the value associated with a key

```
In [72]: # Using D.items() method
print(f"Key-value pairs of the 'rooms' dictionary are rooms.items():\n{rooms.items()}"

# Using D.get(key) method
print(f"Values associated with the key 'Kitchen' can be retrieved using room
```

Key-value pairs of the 'rooms' dictionary are rooms.items():
dict_items([('Hall', {'south': 'Kitchen', 'item': 'key'}), ('Kitchen', {'north': 'Hall', 'item': 'monster'}), ('Dining Room', {'west': 'Hall', 'item': 'potion'})])

Values associated with the key 'Kitchen' can be retrieved using rooms.get('Kitchen'):
{'north': 'Hall', 'item': 'monster'}

Dictionary Methods

- `D.update(D1)` adds the key-value pairs of dictionary D1 to dictionary D

```
In [73]: D1 = {
    'Balcony': {
        'east': 'Hall',
        'degree': 60,
        'item': 'chair'}}# Create a dictionary D1
rooms.update(D1) # Update the 'rooms' dictionary with new data
# Display both D1 and the updated 'rooms' dictionary
print(f"D1 = {D1}")
print(f"The command rooms.update(D1) updates the 'rooms' dictionary to: \n{rooms}")
```

D1 = {'Balcony': {'east': 'Hall', 'degree': 60, 'item': 'chair'}} results in:
{'Balcony': {'east': 'Hall', 'degree': 60, 'item': 'chair'}}

The command rooms.update(D1) updates the 'rooms' dictionary to:
{'Hall': {'south': 'Kitchen', 'item': 'key'}, 'Kitchen': {'north': 'Hall', 'item': 'monster'}, 'Dining Room': {'west': 'Hall', 'item': 'potion'}, 'Balcony': {'east': 'Hall', 'degree': 60, 'item': 'chair'}}

Retrieving Values from Dictionary Using Keys and Indexing

- To retrieve a value, you can also use Indexing with key
 - `D[key]`
- If the key doesn't exist, a `KeyError` will be raised.

```
In [74]: mydic = {'Howard': 'aerospace engineer', 'Leonard': 'physicist', 'Sheldon':
          'Penny': 'waitress', 'Raj': 'astrophysicist'}
print(f"mydic = {'Howard': 'aerospace engineer', 'Leonard': 'physicist', 'S
print(f"Retrieving value for 'Howard' using mydic['Howard'] results in: {myc
```

```
mydic = {'Howard': 'aerospace engineer', 'Leonard': 'physicist', 'Sheldon':
'physicist', 'Penny': 'waitress', 'Raj': 'astrophysicist'}
Retrieving value for 'Howard' using mydic['Howard'] results in: aerospace en
gineer
```

Comparing Dictionaries with Lists

- Similarities
 1. Both allow for heterogeneous types, which can be nested
 2. Both dictionaries and lists enable for the retrieval
 - of an element via its unique index in case of lists (and strings)
 - of a value via its unique key in case of dictionaries
 3. Both can be modified
 - Lists can be modified via the `append()` and `insert()` methods
 - Dictionaries can be modified via the `update()` method

Python List Methods Exercises

Exercise 1: Basic Append

Start with the list `[1, 2, 3]`. Append the integer `4` to it.

Exercise 2: Insert at Index

Start with the list `["apple", "banana", "cherry"]`. Insert the fruit "orange" at index 2.

Exercise 3: Remove Element

Start with the list `["red", "blue", "green"]`. Remove the color "red" from it.

Exercise 4: Extend List

Start with two lists: `["cat", "dog"]` and `["fish", "bird"]`. Extend the first list by adding all the elements from the second list.

Exercise 5: Sort Numbers

Start with the list `my_list5 = [9, 3, 6, 1, 7]`. Create a new list with the sorted elements of `my_list5`. Print both lists

Exercise 6: Sort Strings

Start with the list `["Apple", "banana", "Cherry", "apple"]`. Sort them in

lexicographical order, notice the order of precedence between upper-case and lower-case letters.

Exercise 7: Insert and Remove

Start with the list `["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]`. Insert "Sunday" at index 0 and remove "Wednesday".

Exercise 8: Extend and Sort

Start with two lists: `[12, 4, 8]` and `[15, 3, 7]`. Extend the first list with the second list and sort the resulting list in ascending order.

Exercise 9: Append, Insert, and Sort

Start with the list `[2, 5, 1]`. Append the number `10`, insert the number `100` at index 2, and modify the list so to be sorted in ascending order.

Exercise 10: Comprehensive Exercise

Start with the list `["Alice", "Bob", "Charlie"]`. Append the names "David" and "Eve", remove "Bob", insert "Frank" at index 3, extend the list with another list `["Grace", "Hannah"]`, and sort the final list in lexicographical order.

Exercise 11: Advanced List Manipulation

1. Start with the list `my_list11 = ["apple", "banana", "cherry", "apple", "banana", "apple"]`.
2. Use `sort()` and observe what happens when you have the same element at different positions.
3. Use `insert()` to add "orange" at index 3. Observe how the elements shift. Is the new element where you were expecting it?
4. Extend `my_list11` with the list `[1, 2, 3]` using the operator `+` and print the result. Then see if `my_list` has changed. Do the same using the `extend()` method.
5. Use `my_list11.append([4, 5, 6])` and observe how it differs from `my_list11.extend([4, 5, 6])`.
6. Use `my_list11.append("grape")` and then `my_list11.remove("grape")`. Observe that the last element is removed.
7. redefine `my_list11 = ["apple", "banana", "cherry", "apple", "banana", "apple"]`. Use `my_list11.append(my_list11)` and observe what happens. Try to extract the first element from the last element of the list. Now just extract the last element from `my_list11`. What do you think is happening?
8. Use `my_list11.sort(reverse=True)` to sort the list in descending order.
9. Use `my_list11.pop()` to remove the last element from the list and observe the returned value.

Exercise: Loops Using Control Flow and Indexing

1. Given the list of debts amount `debt_list = [2000, 30000, 4000, 50000, 6000]`, write a loop that prints the outstanding debt voices of a company. Write an informative fstring to explain what is printed.
2. Write another loop that computes and prints the interest cost of each debt entry. The interest rate applied is of the 10% if the debt is below 15000 and 5% if the debt is greater or equal to 15000. Write an informative fstring to explain what is printed.
Hint: use the `if` and `elif` operators.
3. Write another loop that that prints: the original debt, the debt+interests and the index of the debt voice. Write an informative fstring to explain what is printed. **Hint: you will need to use the `range()` and `len()` functions or the `enumerate()` one.**
4. Augment the previous loop by storing the total debt (including interests) in a new list called `debt_list_plusInt`. Print the result with an informative fstring. **Hint: you can either pre-allocate a list of the expected length or use `append`.**
5. Finally, modify the previous loop to store the total debt (including interests) in place of the `debt_list` entries. Print the result with an informative fstring.

Exercise: Analyzing Product Sales

You have three lists: - `products`: A list of product names. - `sales`: A list of the number of units sold for each product. - `prices`: A list of the price per unit for each product.

```
products = ['Apple_Laptop', 'Apple_Mobile', 'Apple_Tablet',
'Samsung_Laptop', 'Samsung_Mobile', 'Samsung_Tablet',
'Sony_Laptop', 'Sony_Mobile', 'Sony_Tablet', 'Dell_Laptop',
'Dell_Mobile', 'Dell_Tablet', 'HP_Laptop', 'HP_Mobile',
'HP_Tablet', 'Nokia_Laptop', 'Nokia_Mobile', 'Nokia_Tablet',
'Asus_Laptop', 'Asus_Mobile', 'Asus_Tablet', 'Acer_Laptop',
'Acer_Mobile', 'Acer_Tablet', 'LG_Laptop', 'LG_Mobile',
'LG_Tablet', 'Lenovo_Laptop', 'Lenovo_Mobile', 'Lenovo_Tablet']
sales = [70, 76, 185, 209, 227, 160, 118, 250, 217, 255, 253, 57,
67, 184, 77, 184, 204, 253, 163, 184, 85, 289, 297, 201, 188, 232,
138, 138, 183, 111]
prices = [1124, 1450, 303, 413, 575, 1438, 360, 400, 380, 596,
1400, 347, 476, 1380, 428, 448, 1446, 977, 1118, 667, 1042, 449,
1199, 1321, 605, 1239, 375, 322, 540, 727]
```

Given three lists:

- `products`: A list of product names.
- `sales`: A list of the number of units sold for each product.
- `prices`: A list of the price per unit for each product.

1. Product Revenue Display:

- Loop through the products, sales, and prices.
- For each product, compute its total revenue (sales * price).
- Print an f-string that details the product's name and its total revenue.

2. Revenue Analysis:

- Using the same loop from step 1, store the computed revenues for each product in a new list.
 - a. **Total Revenue:** Calculate and print the overall total revenue from all products.
 - b. **Top 5 Performers:** Calculate the combined revenue of the top 5 products with the highest revenues.
 - c. **Mid-tier Performers:** Calculate the combined revenue of products ranked 20th to 30th in terms of revenue.
 - d. **Bottom 20 Performers:** Calculate the combined revenue of the 20 products with the lowest revenues.
 - e. **Revenue Extremes:** Identify and print the names of the products with the absolute lowest and highest revenues.
 - f. **Price Extremes:** Identify and print the names of the products with the absolute lowest and highest prices and identify their revenues.
 - g. **Revenue Percentages:** For the revenues computed in sub-tasks a to f, calculate and print their percentages relative to the overall total revenue.

3. Use of `zip` Function:

- If you haven't used the `zip` function in the previous steps, modify your code to utilize it for iterating through the products, sales, and prices simultaneously.